



PUC Minas

# Plataformas Node.js

Samuel Martins



PUC Minas

# Estratégias de Cache

# Estratégias de Cache

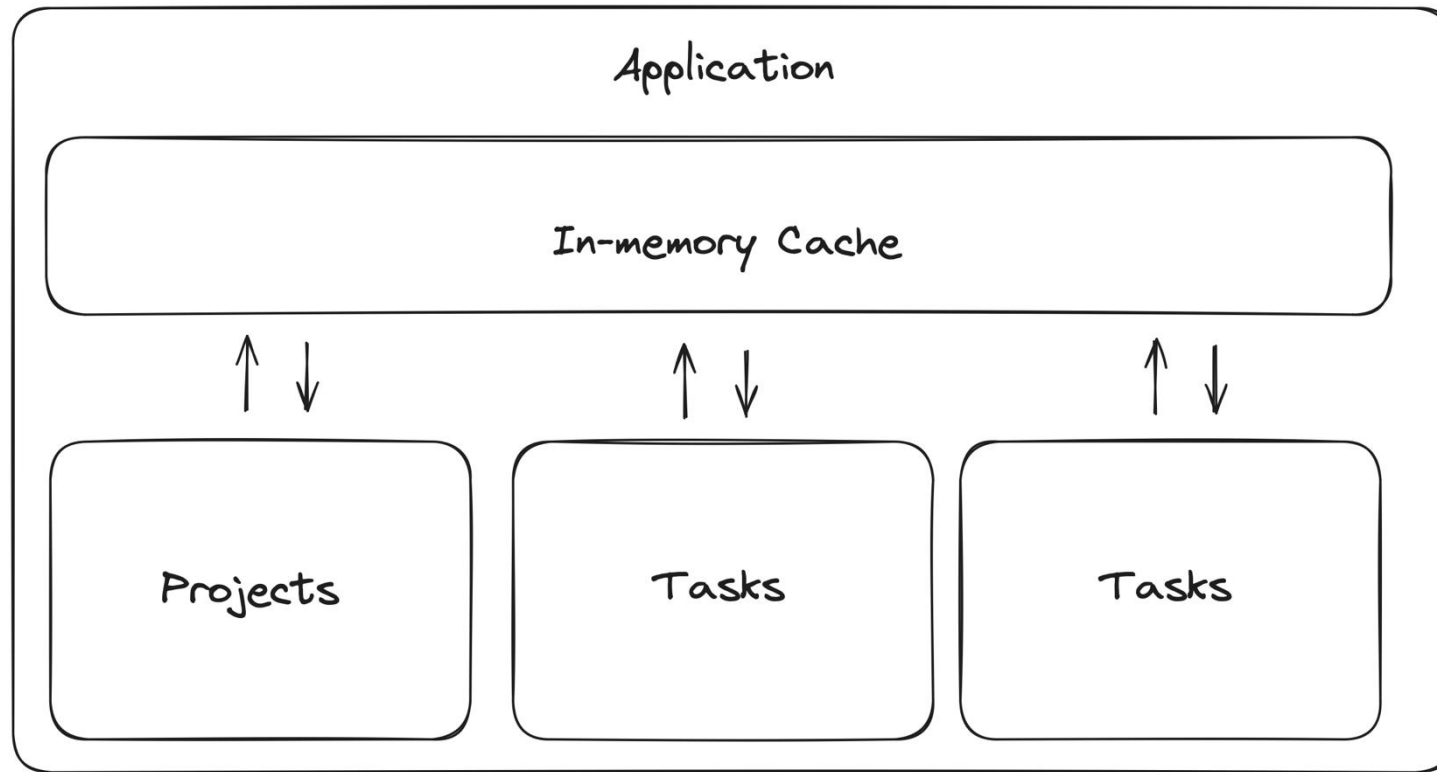
- Cache é uma técnica de armazenamento temporário de dados frequentemente acessados para reduzir a latência e melhorar o desempenho do aplicativo.
- No NestJS, o sistema de cache permite armazenar resultados de consultas ou operações custosas em memória para acesso rápido.

```
$ npm install @nestjs/cache-manager cache-manager
```

# Estratégias de Cache

- O NestJs possui um módulo de cache extensível que pode ser implementado nas nossas aplicações;
- Os métodos do módulo de cache são abstraídos, portanto podemos utilizar algumas stores para o nosso sistema de cache, como:
  - In-memory;
  - Redis;
  - Mongo;
  - [E vários outros sistemas distribuídos.](#)

# Estratégias de cache – In Memory



Fonte: próprio autor

# Estratégias de cache – In Memory

```
import { Module } from '@nestjs/common';  
import { CacheModule } from '@nestjs/cache-manager';  
import { AppController } from './app.controller';  
  
@Module({  
  imports: [CacheModule.register({ isGlobal: true })],  
  controllers: [AppController],  
})  
export class AppModule {}
```

Fonte: próprio autor

# Estratégias de Cache

- O Cache pode ser definido de forma por controller ou por endpoint via decorators de forma automática

```
@Get()
@UseInterceptors(CacheInterceptor)
@CacheTTL(30000)
async findAll() {
    return this.projectsService.findAll();
}
```

# Estratégias de Cache

- Podemos também utilizar o cache manager service para customizarmos a forma como armazenamos o cache por meio de uma lógica específica



# Estratégias de Cache

```
@Controller("projects")
export class ProjectsController {
  constructor(
    private readonly projectsService: ProjectsService,
    @Inject(CACHE_MANAGER) private cacheManager: Cache,
  ) {}

  @Get()
  async findAll(@Query() filter?: FilterDto) {
    const projectsCache = this.cacheManager.get("projects");

    if (projectsCache) {
      return projectsCache;
    }

    const results = await this.projectsService.findAllPaginated(filter);
    this.cacheManager.set("projects", results);
    return results;
  }
}
```

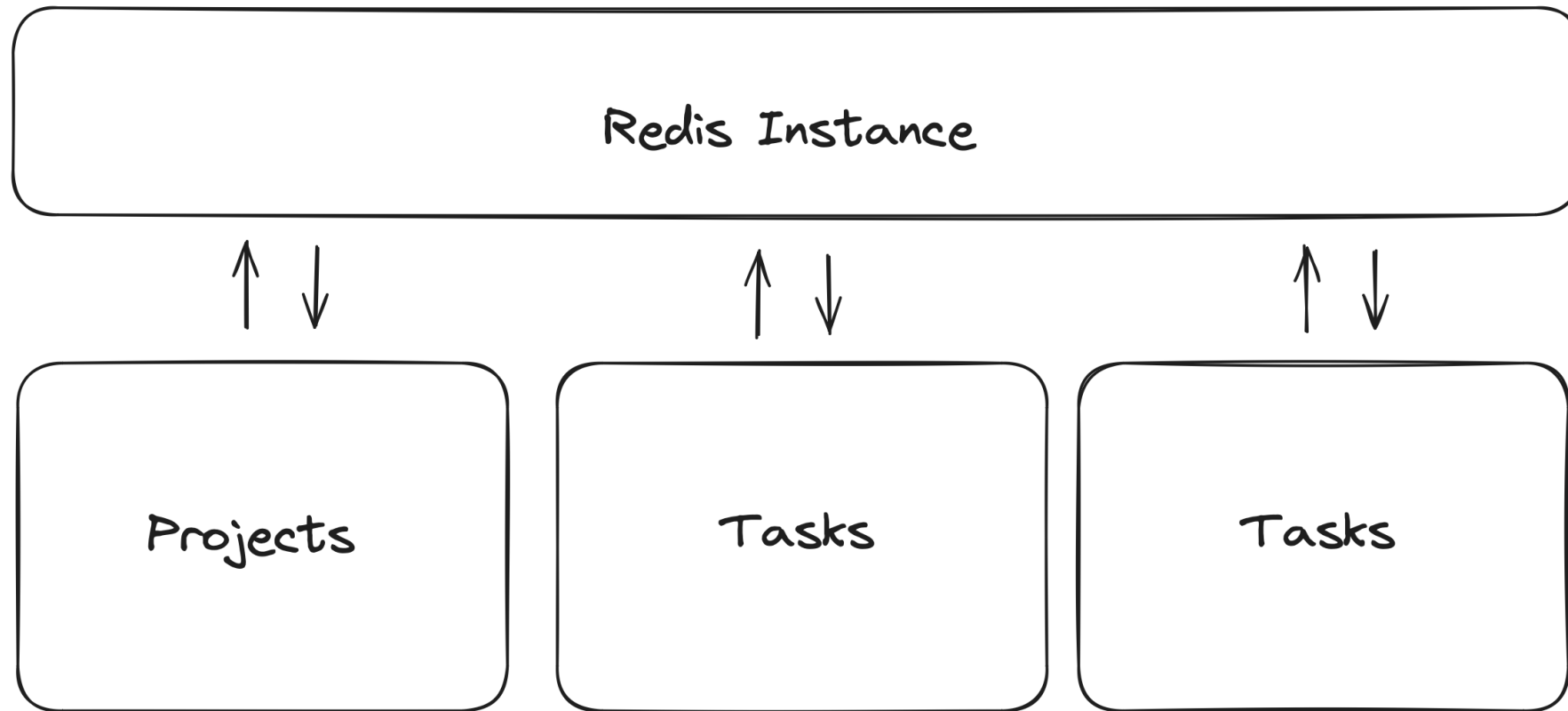
# Estratégias de Cache

- Para implementação do cache via redis, podemos utilizar as opções da instância do CacheModule e passar as referências para o host onde o redis encontra-se hospedado

```
1 CacheModule.register({  
2     isGlobal: true,  
3     store: redisStore,  
4     host: process.env.REDIS_HOST,  
5     port: process.env.REDIS_PORT,  
6 },
```

Fonte: próprio autor

# Estratégias de cache – Redis



Fonte: próprio autor



PUC Minas

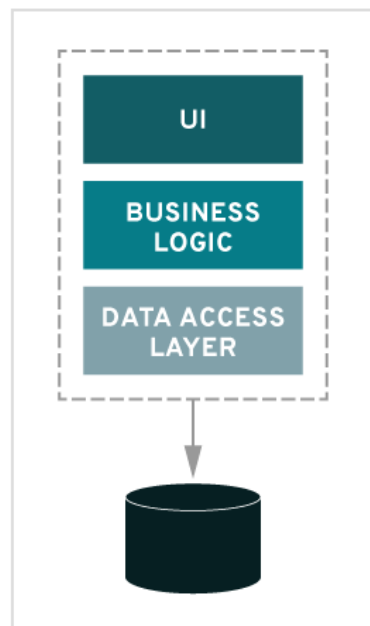
# Micros Serviços

# Micros Serviços

- O conceito de micros serviços em aplicações Node veio para facilitar a operação e escala de softwares de grande porte;
- Implementações de micros serviços adicionam muita complexidade ao sistema como um todo:
  - Desenvolvimento de cada micro serviço precisa de ser pensado no auto-funcionamento (mínimo de dependência possível com outros micro serviços);
  - Troca de informações entre micro-serviços:
  - Manutenção de estado compartilhado e troca de mensagens;

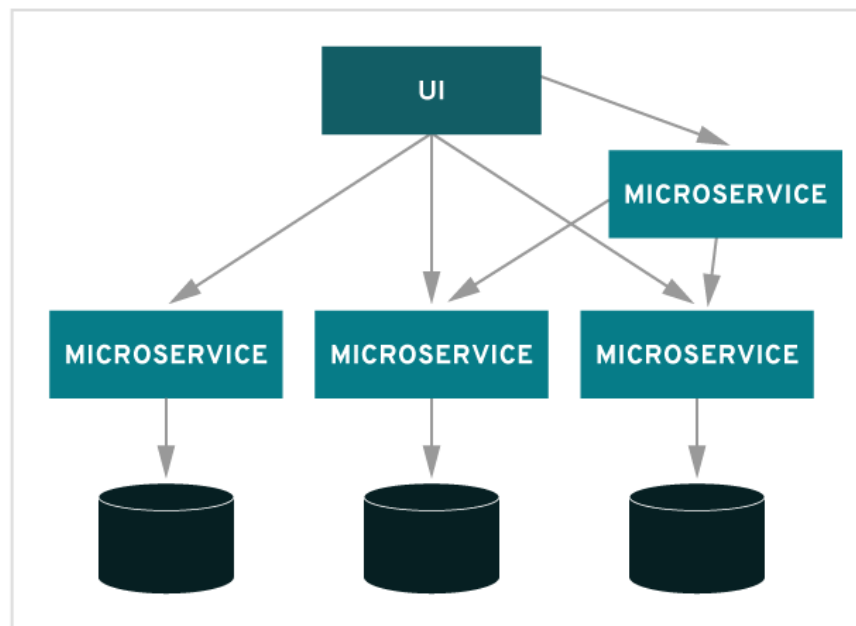
# Micros Serviços

MONOLITHIC



VS.

MICROSERVICES



Fonte: <https://www.redhat.com/pt-br/topics/microservices/what-are-microservices>

# Micros Serviços - Benefícios

- **Escala técnica:** possibilidade de escalar fisicamente (recursos de máquina) serviços que demandam mais processamento e fluxo de dados;
  - **Maior resiliência:** se uma parte do sistema para de funcionar, existe uma menor chance das demais áreas serem afetadas;
  - **Independência de stacks:** embora estejamos falando da stack Node/Js, uma arquitetura de micro serviços permite o sistema a ter uma liberdade de escolha entre tecnologias.

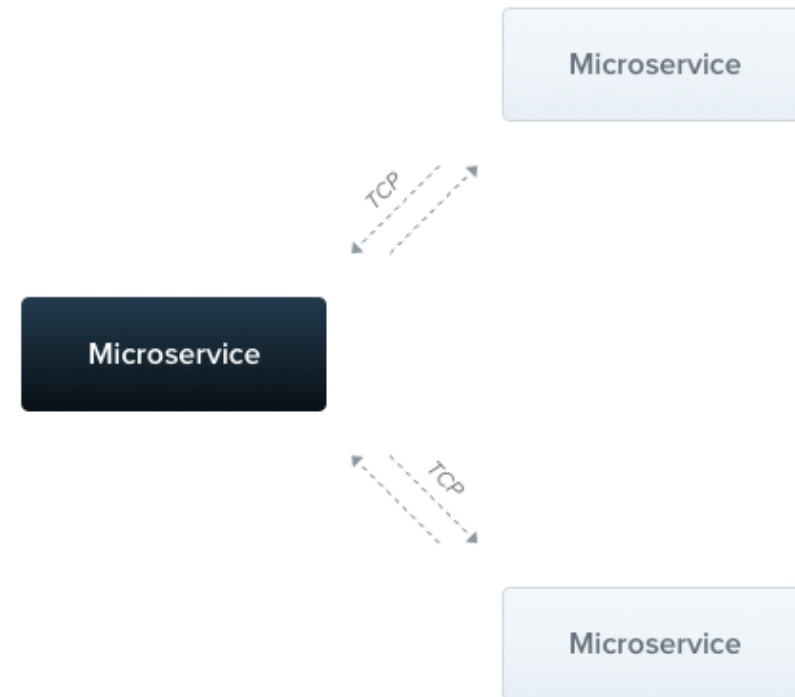
# Micros Serviços - Benefícios

- **Escala organizacional:** possibilidade de segregar o desenvolvimento de serviços por times
  - Cada time pode ficar responsável pela manutenção de um serviço/domínio específico;
  - Os deploys podem ser separados por times, sem criar uma dependência entre as fases dos deploys



# Micros Serviços – TCP vs Mensageria

- **TCP:** serviços instanciados na mesma rede, onde a comunicação se dá através de um protocolo externo;
- **Protocolos de Mensageria:** serviços comunicam entre si através de plataformas terceirizadas de mensageria, como Kafka e RabbitMq



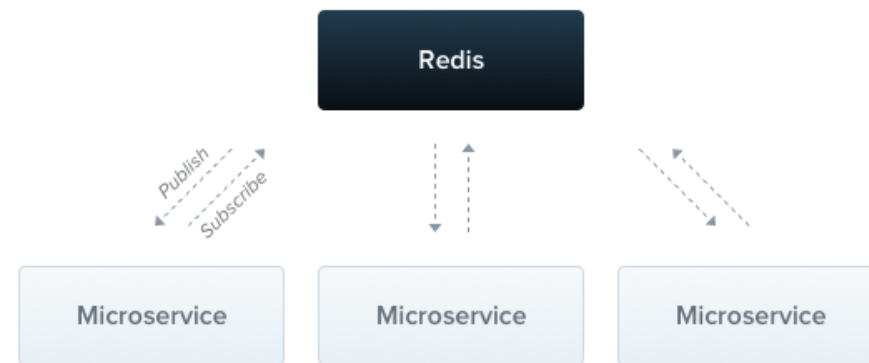
Fonte: <https://docs.nestjs.com/microservices/basics>

# Micros Serviços – TCP vs Mensageria

- A **implementação via TCP** é ***simples e direta*** entre os micro serviços. Se um micro serviço precisa de enviar/solicitar alguma informação para um outro, podemos disparar uma requisição que trafega sob o TCP e chega diretamente do outro lado
  - Possui eventualmente **problemas de escalabilidade** uma vez que se um micro serviço recebe um tráfego alto de requisições, estas podem criar um gargalo na aplicação, congestionando o sistema com o processamento de dados em larga escala.

# Micros Serviços – TCP vs Mensageria

- A implementação via Sistemas de Mensageria é ***robusta e mais complexa*** quando comparada ao modelo TCP. Isso porque um sistema de mensageria exige um segundo sistema que será responsável por receber e enviar mensagens



Fonte: <https://docs.nestjs.com/microservices/redis>

# Micro Serviços – Mensageria

- Os sistemas de mensageria possuem os seguintes conceitos:
  - **Publish:** ato de publicar uma mensagem no sistema de mensageria;
  - **Subscriber:** micro serviço que assina mensagens em um determinado tópico;
  - **Producer:** parte da nossa implementação que produz mensagens a serem enviadas à plataforma;
  - **Consumer:** parte da nossa implementação que consome mensagens produzidas pelos producers e encaminhadas pelo sistema de mensageria.

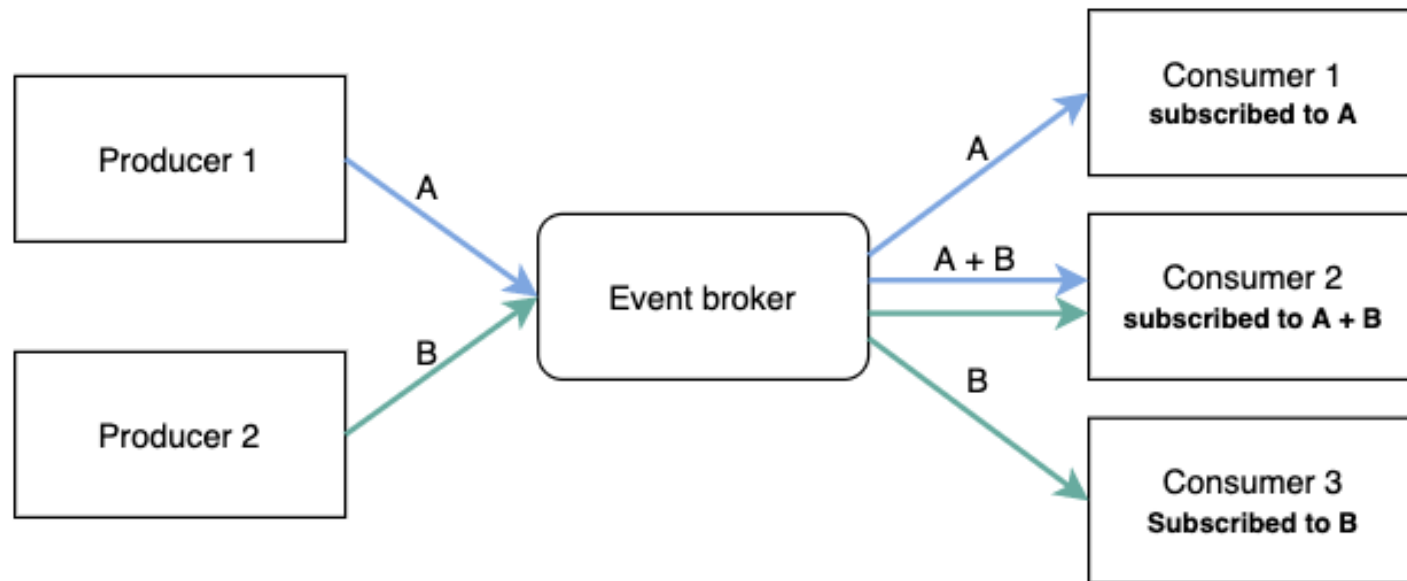
# Micros Serviços – Mensageria

- O padrão **requisição-resposta** é utilizado para troca de mensagens entre micro serviços quando uma requisição feita precisa de uma resposta do serviço de destino;
  - **Uso de dois canais:** um para transferência de dados e outro para esperar respostas;
  - Algumas situações não necessariamente irão exigir uma resposta do serviço de destino, como:
    - Envio de notificações;
    - Disparo de emails.

# Micros Serviços – Mensageria

- O padrão **event-based** é útil quando queremos publicar eventos sem necessariamente precisar esperar por uma resposta;
- Se quisermos notificar o ecossistema que uma determinada condição aconteceu em um dos micro serviços, esse modelo é ideal porque podemos escutar por eventos em múltiplas áreas de micro serviços diferentes, sem necessariamente a simulação de um protocolo requisição-resposta (como HTTP);
- Esse tipo de sistema baseado em eventos é escalável e seu uso pode ser combinado com sistemas de mensageria, como RabbitMQ, Kafka ou Redis.

# Micros Serviços



Fonte: <https://blog.hubspot.com/marketing/event-driven-architecture>



**PUC Minas**